The Journal of Open Source Software

FTorch: a library for coupling PyTorch models to Fortran

Jack Atkinson [1¶], **Athena Elafrou** [2], **Elliott Kasoar** [1,3], **Joseph G. Wallwork** [1], **Thomas Meltzer** [1], **Simon Clifford** [1], **Dominic Orchard** [1,4], **and Chris Edsall** [1]

1 Institute of Computing for Climate Science, University of Cambridge, UK **2** NVIDIA, UK **3** Scientific Computing Department, Science and Technology Facilities Council, UK **4** University of Kent, UK ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 06 December 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

In the last decade, machine learning (ML) and deep learning (DL) techniques have revolutionised many fields within science, industry, and beyond. Researchers across many domains are increasingly seeking to combine ML approaches with traditional numerical modelling to advance research. This typically brings about a computational challenge of *programming language interoperation*. PyTorch is a popular framework for designing and training ML/DL models whilst Fortran remains a language of choice for many scientific models running on high-performance computing (HPC) systems. The `FTorch` library provides an easy-to-use, performant method for coupling the two, allowing users to call PyTorch models from Fortran.

`FTorch` is open-source, open-development, and well documented with minimal dependencies. A central tenet of its design, in contrast to other approaches, is that FTorch removes dependence on the Python runtime (and virtual environments). By building on the `LibTorch` backend it allows users to run ML models on both CPU and GPU architectures without the need for porting code to device-specific languages.

Statement of need

The explosion of ML/DL has brought several promising opportunities to deploy these techniques in scientific research. There are notable applications in the physical sciences (Carleo et al., 2019), climate science (Kashinath et al., 2021), and materials science (Bishara et al., 2023). Common applications include the emulation of computationally intensive processes and the development of data-driven components. Such deployments of ML can achieve higher computational performance and/or improved predictive performance, compared to traditional numerical techniques. A common example from the geosciences is the development of ML parameterisations of subgrid processes - currently a major source of uncertainty in many models.

Fortran is widely used for scientific codes due to its performance, stability, array-oriented design, and native support for shared and distributed memory, amongst other features (Kedward et al., 2022). Many ML frameworks, on the other hand, are accessed using Python. A commonly-used framework is PyTorch (Paszke et al., 2019) which allows users to design and deploy ML models with many advanced features.

Ideally users would develop and validate ML models in the PyTorch environment before deploying them into a scientific model. This deployment should be possible with minimal additional code, and guarantee identical results to those obtained with the PyTorch interface –

something not guaranteed if re-implementing by hand in Fortran. Ideally one would call out, from Fortran, to an ML model saved from PyTorch, with the results returned directly to the scientific code.

`FTorch` bridges this gap, reducing the burden on researchers seeking to incorporate ML into their numerical models. It provides an intuitive and user-friendly interface from Fortran to ML models developed using PyTorch. It removes the need for detailed knowledge about language interoperation and the need to develop bespoke coupling code, instead providing a Fortran interface designed to be familiar to both PyTorch and Fortran users.

Further, `FTorch` has no dependence on the Python runtime. This is appealing for deployment on HPC systems where the management of Python environments can be challenging, and it avoids the incurred overhead of object representations in Python's runtime.

Software description

PyTorch is itself built on an underlying framework `LibTorch` written in `C++` which can be obtained as a separate library and is accessible through a `C++` API. By accessing this directly (rather than via PyTorch), `FTorch` avoids the use of Python at run-time.

Using the `iso_c_binding` module, intrinsic to Fortran since the 2003 standard, we provide a Fortran wrapper to `LibTorch`. This enables shared memory use (where possible) to further improve efficiencies by reducing data-transfer during coupling.[1]

`FTorch` is open source on GitHub. It can be built from source using CMake. Minimum dependencies are an installation of `LibTorch`, CMake, and Fortran (2008 standard), `C`, and `C++` (`C++17` standard) compilers.[2] The library is primarily developed in Linux, but also runs on macOS (arm64) and Windows.

Key components and workflow leveraging FTorch

1. Build, train, and validate a model in PyTorch.
2. Save the model in TorchScript form, a strongly-typed subset of Python.
3. Write Fortran code using the `FTorch` Fortran module to:
   - load the TorchScript model;
   - create Torch tensors from Fortran arrays;
   - run the model for inference;
   - use the returned data as a Fortran array;
   - deallocate any temporary FTorch objects;
4. Compile the Fortran code, linking to the FTorch installation.

PyTorch tensors are represented by `FTorch` as a `torch_tensor` derived type, and can be created from Fortran arrays using the `torch_tensor_from_array()` subroutine. Tensors are supported across a range of data types up to rank 5[3] using the fypp preprocessor (Aradi, 2024)

Building on top of `LibTorch`, we utilise existing support for GPU acceleration without additional device-specific code. To target a `torch_tensor` to a device, users specify the corresponding `device_type` enum – `torch_kCPU`, `torch_kCUDA` etc. with targeting multiple GPUs through the optional `device_index` argument. The device types available can be extended as `LibTorch` adds support for new devices.

TorchScript models are represented by the `torch_model` derived type to which saved models can be loaded using the `torch_model_load()` subroutine. Models can be run for inference

[1] i.e. the same data in memory is used by both `LibTorch` and `Fortran` without creating a copy.

[2] To utilise GPU devices, users require the appropriate `LibTorch` binary plus any relevant dependencies, e.g. CUDA for NVIDIA devices.

[3] This is easily extendable at build time for users who require more dimensions.

DRAFT

using the `torch_model_forward()` subroutine with input and output `torch_tensors` supplied as arguments. Finally, FTorch types can be deallocated using `torch_delete()`.

The following provides a minimal example:

```fortran
use ftorch
...
type(torch_model) :: model
type(torch_tensor), dimension(n_inputs)  :: model_inputs
type(torch_tensor), dimension(n_outputs) :: model_outputs
...
call torch_model_load(model, "/path/to/saved_TorchScript_model.pt")
call torch_tensor_from_array(model_inputs(1),  fortran_inputs,  &
                             in_layout, torch_kCPU)
call torch_tensor_from_array(model_outputs(1), fortran_outputs, &
                             out_layout, torch_kCPU)
...
call torch_model_forward(model, model_inputs, model_outputs)
...
call torch_delete(model)
call torch_delete(model_inputs)
call torch_delete(model_outputs)
...
```

FTorch's webpage can be found at https://cambridge-iccs.github.io/FTorch. It contains a user guide, API documentation, slides and videos for presentations on FTorch, and links to projects using FTorch.

Examples and Tooling

Included in the library is a directory of documented examples which demonstrate calling a net from both Python and Fortran. These cover basic use, running with multiple inputs/outputs, using (multiple) GPU devices, and structuring code for deployment in scientific applications.

These examples underpin the testing suite which performs a series of integration tests running as part of a Continuous Integration workflow over various platforms and compilers. Other components include fypp templating checks and code quality checks using fortitude (Pattinson & Hill, 2024) for Fortran, clang-format (The Clang Team, 2024a) and clang-tidy (The Clang Team, 2024b) for `C` and `C++`, and ruff (Astral, 2024) for Python.

The library also provides a script (`pt2ts.py`) to assist users with saving their PyTorch models to TorchScript.

Comparison to other approaches

- **Replicating a net in Fortran**
  That is, a user takes a model developed and trainied in PyTorch and reimplements it from scratch using only Fortran, loading weights saved from PyTorch. This is likely to require considerable development effort, re-writing code that already exists and missing opportunities to use the diverse and highly-optimised features of Torch. Such a re-implementation can be a source of error, requiring additional testing to ensure correctness of the re-implemented model.
  If the overall goal is simply to incorporate ML into Fortran, rather than using PyTorch specifically, then another approach is to leverage a Fortran-based ML framework such as neural-fortran (Curcic, 2019). Whilst it does not allow interaction with PyTorch, neural-fortran provides many neural network components for building nets directly in Fortran. However, the set of features is not as rich as PyTorch and GPU offloading is

DRAFT

not currently supported. The Fiats ('Functional Inference And Training for Surrogates') package is another approach for developing, training, and deploying ML models directly in Fortran, with experimental GPU support at present.

- **Forpy** (Rabel, 2020)
  Forpy is a Fortran module that provides access to Python data structures (including `numpy` arrays) for interoperability. Whilst this provides wider access to general Python features it has a challenging interface with more boilerplate. It also requires access to the Python runtime from Fortran.

- **TorchFort** (NVIDIA, 2024) Since we started our work on `FTorch` a similar library has been released by NVIDIA. This has a similar approach to `FTorch`, in that it avoids Python and links against the `LibTorch` backend. It has a focus on enabling GPU deployment on NVIDIA hardware.

- **SmartSim** (Partee et al., 2022)
  SmartSim is a workflow library developed by HPE and built upon Redis API. It provides a framework for launching ML and HPC workloads transferring data between the two via a database. This is a more versatile approach that can work with a variety of languages and ML frameworks. However, it has a significant learning curve, incurs data-transfer overheads, and requires managing tasks from Python.

Examples of Use

In addition to the educational examples included with the library, `FTorch` is actively used in scientific research:

- in the DataWave project to couple a neural net emulation for gravity wave drag to the MiMA atmospheric model (DataWave, 2023) demonstrating variability of models trained offline when coupled to a host (Mansfield & Sheshadri, 2024).

- to couple a U-Net based model of multi-scale convection into ICON (the Icosahedral Non-hydrostatic modelling framework) and demonstrate via Shapley values that non-causal learnt relations are more stable when running online (Heuer et al., 2024).

- in the DataWave project (DataWave, 2024) to couple emulators of gravity wave drag, and new data-driven parameterisations to the Community Atmosphere Model (CAM) running on HPC systems.

- As part of CESM (the Community Earth System Model) working to provide a general approach for researchers to couple ML models to the various components of the model suite.

Future development

Recent work in scientific domains suggests that online training is likely important for long-term stability of hybrid models (Brenowitz et al., 2020). We therefore plan to extend FTorch to expose PyTorch's autograd functionality to support this.

We welcome feature requests and are open to discussion and collaboration.

Acknowledgments

This project is supported by Schmidt Sciences, LLC. We also thank the Institute of Computing for Climate Science for their support.

DRAFT

References

Aradi, B. (2024). *Fypp*. https://fypp.readthedocs.io

Astral. (2024). *Ruff*. https://docs.astral.sh/ruff/

Bishara, D., Xie, Y., Liu, W. K., & Li, S. (2023). A state-of-the-art review on machine learning-based multiscale modeling, simulation, homogenization and design of materials. *Archives of Computational Methods in Engineering*, *30*(1), 191–222. https://doi.org/10.1007/s11831-022-09795-8

Brenowitz, N. D., Henn, B., McGibbon, J., Clark, S. K., Kwa, A., Perkins, W. A., Watt-Meyer, O., & Bretherton, C. S. (2020). Machine learning climate model dynamics: Offline versus online performance. *arXiv Preprint arXiv:2011.03081*. https://doi.org/10.48550/arXiv.2011.03081

Carleo, G., Cirac, I., Cranmer, K., Daudet, L., Schuld, M., Tishby, N., Vogt-Maranto, L., & Zdeborová, L. (2019). Machine learning and the physical sciences. *Reviews of Modern Physics*, *91*(4), 045002. https://doi.org/10.1103/RevModPhys.91.045002

Curcic, M. (2019). A parallel Fortran framework for neural networks and deep learning. *ACM SIGPLAN Fortran Forum*, *38*, 4–21. https://doi.org/10.1145/3323057.3323059

DataWave. (2023). *MiMA machine learning*. https://github.com/DataWaveProject/MiMA-machine-learning

DataWave. (2024). *DataWave CAM-GW*. https://github.com/DataWaveProject/CAM

Heuer, H., Schwabe, M., Gentine, P., Giorgetta, M. A., & Eyring, V. (2024). Interpretable multiscale machine learning-based parameterizations of convection for ICON. *Journal of Advances in Modeling Earth Systems*, *16*(8), e2024MS004398. https://doi.org/10.1029/2024MS004398

Kashinath, K., Mustafa, M., Albert, A., Wu, J., Jiang, C., Esmaeilzadeh, S., Azizzadenesheli, K., Wang, R., Chattopadhyay, A., Singh, A., & others. (2021). Physics-informed machine learning: Case studies for weather and climate modelling. *Philosophical Transactions of the Royal Society A*, *379*(2194), 20200093. https://doi.org/10.1098/rsta.2020.0093

Kedward, L. J., Aradi, B., Čertík, O., Curcic, M., Ehlert, S., Engel, P., Goswami, R., Hirsch, M., Lozada-Blanco, A., Magnin, V., & others. (2022). The state of Fortran. *Computing in Science & Engineering*, *24*(2), 63–72. https://doi.org/10.1109/MCSE.2022.3159862

Mansfield, L. A., & Sheshadri, A. (2024). Uncertainty quantification of a machine learning subgrid-scale parameterization for atmospheric gravity waves. *Journal of Advances in Modeling Earth Systems*, *16*(7), e2024MS004292. https://doi.org/10.1029/2024MS004292

NVIDIA. (2024). *TorchFort*. https://nvidia.github.io/TorchFort/

Partee, S., Ellis, M., Rigazzi, A., Shao, A. E., Bachman, S., Marques, G., & Robbins, B. (2022). Using machine learning at scale in numerical simulations with SmartSim: An application to ocean climate modeling. *Journal of Computational Science*, *62*, 101707. https://doi.org/10.1016/j.jocs.2022.101707

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., & others. (2019). Pytorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, *32*.

Pattinson, L., & Hill, P. (2024). *Fortitude*. https://github.com/PlasmaFAIR/fortitude

Rabel, E. (2020). *Forpy*. https://github.com/ylikx/forpy

The Clang Team. (2024a). *Clang-format*. https://clang.llvm.org/docs/ClangFormat.html

DRAFT

The Clang Team. (2024b). *Clang-tidy*. https://clang.llvm.org/extra/clang-tidy/

DRAFT